ReasonLink: Characterizing and Optimizing the Handoff Between Neural and Symbolic Accelerators

Vayun Gupta (vg2684), Aaron Mano Cherian (amc2535)
COMS 6998: AI-Native Computing | Fall 2026
Project Proposal | Due October 5, 2026, 11:59 PM ET

Track / topic: A: Computing for AI; Computing Systems for Neuro-Symbolic and Compositional AI.

I. Problem and Motivation

Neuro-symbolic systems pair dense neural perception with irregular symbolic reasoning, and the two stages are best served by different accelerators. Between them sits a *handoff*: neural outputs (e.g., class scores) must be converted into the solver's representation (variables, clauses, or assignments), moved through the memory system, and synchronized with the consumer. This handoff is usually host-CPU software and is rarely measured.

Amdahl's law applies to the glue as well as to the kernels: once both stages are accelerated, a handoff that was negligible next to slow kernels can dominate end-to-end time and leave both accelerators idle [1], [2]. Concretely, SAT-Accel [3] is loaded through a host-prepared memory image of its clause store and per-literal metadata, using a 64-byte record per literal, even when the neural stage contributes only a few bits per variable. Locating handoff time shows which interface mechanisms turn kernel speedups into system speedups.

We therefore ask: *after both ends of a neuro-symbolic pipeline are accelerated, does the interface between them become the bottleneck, and if so, which interface mechanisms remove it?* We will (1) decompose the handoff between Gemmini [4] and SAT-Accel [3] into representation conversion, data movement, and synchronization, and (2) design ReasonLink, an interface that reduces the exposed handoff relative to serial and double-buffered baselines. If the handoff is always amortized, identifying why is itself a result.

II. Related Work

Wan et al. [5] characterize neuro-symbolic workloads on CPUs, GPUs, and edge SoCs and find that symbolic components are memory-bound and often lie on the critical path. CogSys [6] co-designs reconfigurable neural/symbolic processing elements, a streaming dataflow, and a workload-aware scheduler. NSFlow [7] generates FPGA dataflow architectures that run neural and symbolic operators on a shared adaptive array. REASON [8] accelerates probabilistic logical reasoning with a unified graph representation and a multi-level pipeline coupled to a GPU. These systems place both stages in one co-designed fabric, so the handoff is internal to the design. We study the complementary and common case: two *independently designed* accelerators composed in an SoC, where the handoff is explicit and must be engineered.

LogCA [2] models how offload latency and granularity bound accelerator speedup, and Richins et al. [1] show that non-kernel work limits end-to-end AI applications. We apply this view to an accelerator-to-accelerator boundary, where conversion is a semantic translation from tensors to constraints.

Our building blocks are Gemmini [4], a systolic-array DNN accelerator for RISC-V SoCs built with Chipyard, and SAT-Accel [3], a CDCL solver built in Vitis HLS as seven stream-connected kernels. Visual Sudoku follows SATNet.

III. Proposed Approach

We will build one Chipyard SoC containing a Rocket core, Gemmini (attached via RoCC), and SAT-Accel. SAT-Accel's kernels will be exported from Vitis HLS as RTL, reconnected according to their stream topology, and wrapped in Chisel. Their AXI masters will attach to the TileLink system bus, and control registers will be exposed through MMIO. We will reduce SAT-Accel's capacity parameters to fit our workloads. Sudoku runs will use cycle-accurate RTL simulation; larger runs will use FireSim if simulation speed requires it.

Our workload is visual Sudoku: Gemmini runs an int8 MNIST classifier over the given cells; cell positions and blanks are known. The solver receives the standard Sudoku CNF plus the predicted givens as unit literals. The base formula is identical across puzzles; only the givens change.

The *serial* baseline (Fig. 1) follows SAT-Accel's native host flow. Gemmini writes scores to memory; the CPU computes the argmax, builds the complete SAT-Accel memory image for the instance, starts the solver over MMIO, and polls for completion. The *double-buffered* baseline uses the same path with two buffer sets, so inference and conversion for puzzle $i{+}1$ overlap solving of puzzle i. Both baselines use explicit completion and buffer-ownership signals to stay correct under backpressure.

ReasonLink's main idea is to make the handoff aware of data movement. It combines four mechanisms that we will evaluate separately and together: (i) a *compact intermediate representation*: a stream of solver-native literals (variable, polarity, and optional confidence), produced in hardware at Gemmini's output instead of a CPU-built memory image; (ii) *on-chip buffering*: a credit-controlled FIFO that carries this stream directly to SAT-Accel and bypasses DRAM; (iii) *delta*

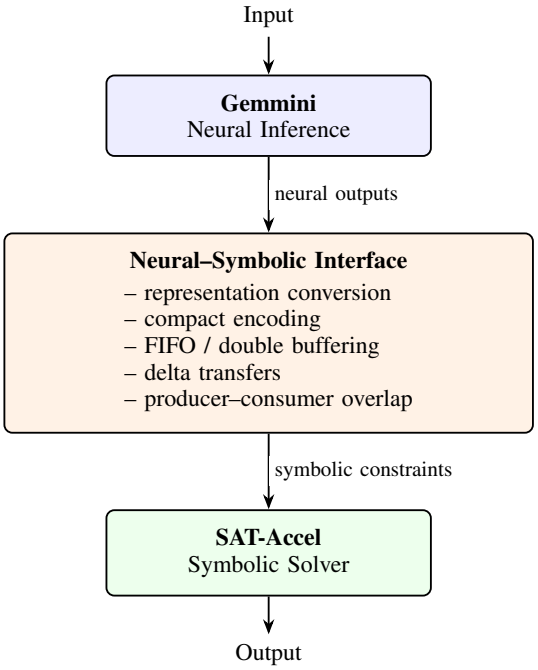


Fig. 1. Neural–symbolic pipeline. Baselines implement the interface stage (orange) in CPU software through shared memory; ReasonLink implements it as a direct hardware stream.

transfers: the base formula stays resident in SAT-Accel, and each instance sends only its changed givens, while learned clauses are rolled back to the base formula between instances; (iv) *producer–consumer overlap*: solver state is loaded as literals arrive, and search starts only once the constraint set is complete. Mechanisms (iii) and (iv) require targeted changes to SAT-Accel's copy-in and reset logic. The expected contribution is a measured handoff breakdown and an interface that reduces it.

IV. Evaluation Plan

We will compare serial, double-buffered, and ReasonLink interfaces on visual Sudoku. All configurations use identical models, predictions, encodings, and solver settings, isolating the interface. Timing runs from inputs in DRAM to the solution in DRAM; cycle counters in each wrapper record busy, stall, and idle intervals.

To quantify the handoff, we decompose a serial run as

$$T_{\text{e2e}} = T_{\text{nn}} + \underbrace{T_{\text{conv}} + T_{\text{xfer}} + T_{\text{sync}}}_{T_{\text{handoff}}} + T_{\text{sat}},$$

and we report the handoff fraction $T_{\text{handoff}}/T_{\text{e2e}}$. For overlapped runs, we report only the exposed handoff, $T_{\text{e2e}} - T_{\text{nn}} - T_{\text{sat}}$, to avoid double counting. We also report latency, throughput (puzzles/s), bytes moved per instance, synchronization and stall cycles, and per-accelerator utilization.

We will test three hypotheses. *H1:* in the serial baseline, the handoff is a large fraction of Sudoku runtime, dominated by CPU conversion and full-image loading rather than by the size of the neural output. *H2:* double buffering hides the handoff only while it is shorter than the slower stage, and moves no fewer bytes. *H3:* ReasonLink's benefit shrinks as T_{sat} grows; we will locate this crossover by varying instance hardness and size.

For ablations, we will add each mechanism individually and cumulatively, and sweep FIFO depth and transfer granularity. To separate algorithmic savings from hardware savings, we will also evaluate delta transfers as a CPU-only optimization. Puzzle accuracy, UNSAT rate (misclassified givens can make a puzzle unsatisfiable), and SAT assignments must be identical across interfaces and are checked against software references. Over-capacity instances and timeouts are reported separately.

V. Timeline and Team Responsibilities

Date	Planned work and deliverables
Oct 19	*Infrastructure and baselines.* Gemmini runs the classifier; SAT-Accel RTL solves Sudoku CNFs in simulation; both baselines run end to end with cycle counters.
Nov 2	*Prototype and pilot results.* Baseline handoff breakdown (H1–H2); ReasonLink prototype with literal stream and on-chip FIFO; Nov 6 midterm talk.
Nov 23	*Initial results.* Delta transfers and overlap complete; three-way comparison across batch sizes and FIFO depths.
Dec 7	*Main results and ablations.* Per-mechanism ablations, crossover analysis (H3); poster for Dec 11.
Dec 18	*Final paper* and reproducible code, RTL, and scripts.

Team's Roles: Both team members will contribute across Gemmini/Chipyard integration, SAT-Accel, the interface, and evaluation. Implementation and experimental tasks will be split for parallel work, while design decisions, debugging, experiments, and analysis remain shared. Regular review of each other's work will ensure both of us maintain a complete understanding of the pipeline.

REFERENCES

[1] D. Richins, D. Doshi, M. Blackmore, A. T. Nair, N. Pathapati, A. Patel, B. Daguman, D. Dobrijalowski, R. Illikkal, K. Long, D. Zimmerman, and V. Janapa Reddi, "Missing the forest for the trees: End-to-end AI application performance in edge data centers," in *IEEE International Symposium on High Performance Computer Architecture (HPCA)*, 2020.

[2] M. S. B. Altaf and D. A. Wood, "LogCA: A high-level performance model for hardware accelerators," in *Proceedings of the 44th Annual International Symposium on Computer Architecture (ISCA)*, 2017.

[3] M. Lo, M.-C. F. Chang, and J. Cong, "SAT-Accel: A modern SAT solver on a FPGA," in *Proceedings of the 2025 ACM/SIGDA International Symposium on Field Programmable Gate Arrays*, 2025, pp. 234–246. [Online]. Available: https://github.com/milo168/FPGA25_SAT_Accel

[4] UC Berkeley Architecture Research, "Gemmini: Full-system, full-stack DNN hardware exploration," Project documentation, accessed October 4, 2026. [Online]. Available: https://github.com/ucb-bar/gemmini/blob/master/README.md

[5] Z. Wan, C.-K. Liu, H. Yang, R. Raj, C. Li, H. You, Y. Fu, C. Wan, S. Li, Y. Kim *et al.*, "Towards efficient neuro-symbolic AI: From workload characterization to hardware architecture," *IEEE Transactions on Circuits and Systems for Artificial Intelligence*, 2024, arXiv:2409.13153.

[6] Z. Wan, H. Yang, R. Raj, C.-K. Liu, A. Samajdar, A. Raychowdhury, and T. Krishna, "CogSys: Efficient and scalable neurosymbolic cognition system via algorithm-hardware co-design," in *2025 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, 2025. [Online]. Available: https://zishenwan.github.io/publication/HPCA25_CogSys.pdf

[7] H. Yang, Z. Wan, R. Raj, J. Park, Z. Li, A. Samajdar, A. Raychowdhury, and T. Krishna, "NSFlow: An end-to-end FPGA framework with scalable dataflow architecture for neuro-symbolic AI," in *Proceedings of the 62nd ACM/IEEE Design Automation Conference (DAC)*, 2025, arXiv:2504.19323.

[8] Z. Wan, C.-K. Liu, J. Qian, H. Yang, A. Raychowdhury, and T. Krishna, "REASON: Accelerating probabilistic logical reasoning for scalable neuro-symbolic intelligence," in *2026 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, 2026. [Online]. Available: https://arxiv.org/abs/2601.20784